# Fascículo de estudio — Programación en Red

## Teoría completa y ejercicios · Preparación del primer parcial

**Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo · Plan 2023 Docente:** Mtro. Jose Pablo Rueda Holguín **Cubre:** Unidad I (Fundamentos) y Unidad II (Sockets) · **Examen primer parcial: 07 de octubre de 2026 Referencias principales:** López-Fuentes (2015), *Sistemas Distribuidos* (Unidad I) · Rhodes & Goerzen (2014), *Foundations of Python Network Programming* (Unidad II)

---

## Cómo usar este fascículo

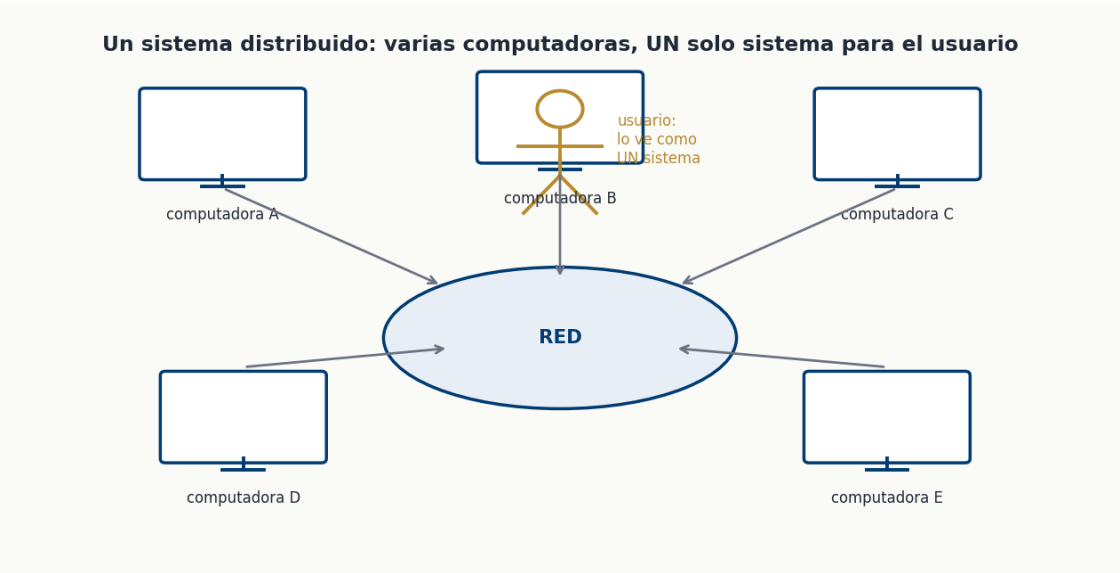
El fascículo está pensado para **imprimirse en dos partes**:

- **Primera parte — Teoría:** todo el contenido, para leer y estudiar. Cada tema sigue la misma estructura: **¿Qué es? → ¿Por qué existe? / ¿Para qué sirve? → ¿Cómo funciona? (con diagrama) → Ejemplo → Idea clave**.
- **Segunda parte — Ejercicios:** formato tipo examen, **con espacios para contestar a lápiz** directamente en la hoja impresa.

# PRIMERA PARTE — TEORÍA

## Tema 1. Sistemas distribuidos

**¿Qué es?** Un **sistema distribuido (distributed system)** es aquel en el que componentes de hardware o software, ubicados en **computadoras conectadas por red**, se comunican y coordinan sus acciones **únicamente mediante paso de mensajes** (Coulouris et al., cap. 1). Dicho de otro modo (Tanenbaum): una colección de computadoras **autónomas** que se presenta ante el usuario como **un solo sistema coherente**.




*Un sistema distribuido*

*Figura 1. Varias computadoras autónomas conectadas por una red; para el usuario, todo funciona como un solo sistema.*

**¿Por qué existen?** Porque los problemas reales así lo exigen:

- **Compartir recursos:** archivos, impresoras, bases de datos, servicios.
- **Escalar:** una sola computadora no da abasto con millones de usuarios.
- **Tolerar fallos:** si una máquina se cae, otra continúa.
- **Ubicuidad:** los usuarios están en distintos lugares y dispositivos.

**Ejemplos cotidianos:** la web, el correo electrónico, la mensajería instantánea, los videojuegos en línea, la banca móvil, el DNS.

**El precio que se paga — problemas que NO existen en una sola máquina:**

- **Latencia (latency):** los mensajes tardan en viajar.
- **Fallos parciales:** puede fallar UNA parte y el resto seguir vivo; tu programa debe decidir qué hacer.
- **Pérdida y desorden:** según el transporte, los mensajes pueden perderse o llegar en distinto orden.
- **Concurrencia (concurrency):** muchos clientes a la vez sobre el mismo servidor.
- **Heterogeneidad:** máquinas, sistemas y lenguajes distintos deben entenderse.

**Idea clave:** programar en red es escribir programas cuyas partes viven en máquinas distintas y colaboran intercambiándose mensajes. Todo el curso son técnicas para lidiar con los problemas de esta lista.

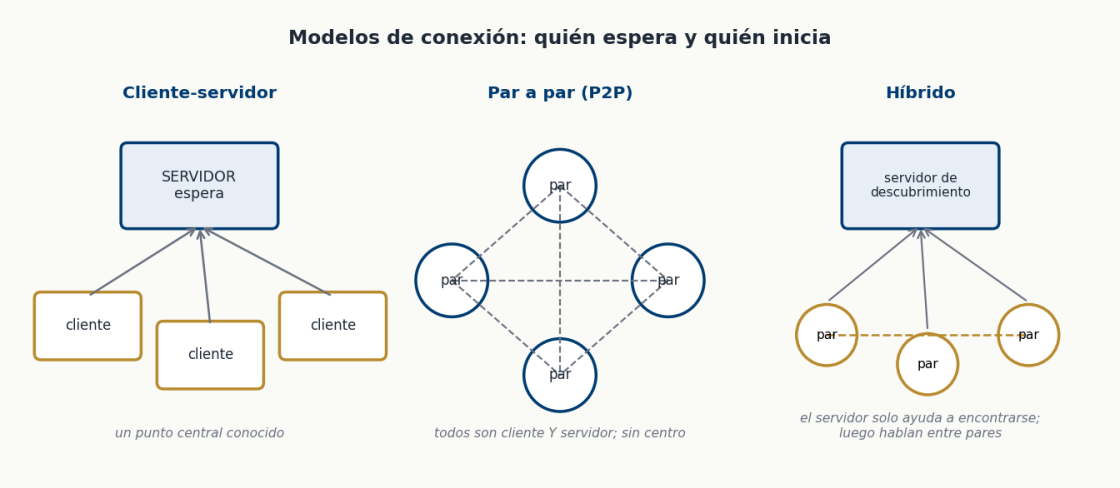
## Tema 2. Modelos de conexión: ¿quién espera y quién inicia?

La primera decisión de diseño de un sistema distribuido es **qué papel juega cada participante** (López-Fuentes, 2015, sec. 4.5). Hay tres modelos:

**Cliente-servidor (client-server).** Dos papeles fijos: el **servidor (server) espera pasivamente** en una dirección conocida; el **cliente (client) inicia** la comunicación y pide; el servidor responde. Existe porque hace falta un punto central **confiable y siempre disponible**. Ejemplos: web, correo, DNS.

**Par a par (peer-to-peer, P2P).** Sin papeles fijos: cada **par (peer)** es **cliente y servidor a la vez**. Existe porque escala sin servidor central costoso. Ejemplos: BitTorrent, compartición de archivos.

**Híbrido (hybrid).** Combina ambos: hay un servidor, pero solo para **descubrirse o coordinarse**; el grueso del intercambio ocurre **entre pares**. Ejemplos: juegos en línea con servidor de emparejamiento, videollamadas con servidor de señalización.




*Modelos de conexión*

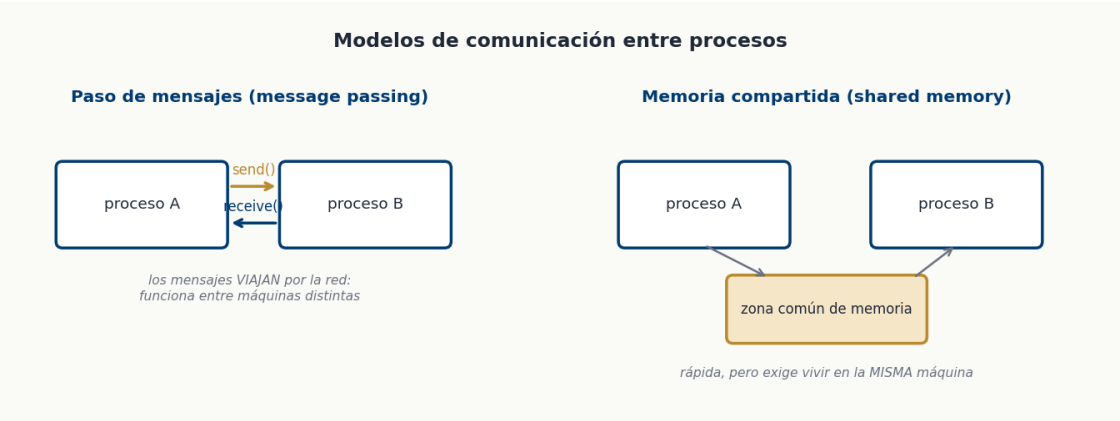
*Figura 2. Los tres modelos: cliente-servidor (un centro conocido), P2P (sin centro) e híbrido (centro solo para encontrarse).*

> **Idea clave:** el modelo responde a "¿quién espera y quién inicia?". En este curso trabajamos sobre todo **cliente-servidor**.

## Tema 3. Modelos de comunicación: ¿cómo se pasan la información?

Decidido QUIÉN habla con quién, la siguiente pregunta es **CÓMO** se comunican los procesos (López-Fuentes, 2015, sec. 4.5):

- **Memoria compartida (shared memory):** los procesos leen y escriben una **zona de memoria común**. Muy rápida, pero exige vivir en la **misma máquina** — no sirve por red.
- **Paso de mensajes (message passing):** los procesos se envían **mensajes explícitos**: uno hace **send (enviar)** y otro **receive (recibir)**. Funciona igual entre procesos locales o entre continentes. **Es EL modelo de la red** — el que implementan los sockets.
- **Llamada a procedimiento remoto (RPC):** evolución del paso de mensajes: en vez de "enviar mensajes", el programa **llama funciones** que se ejecutan en otra máquina (Tema 12).
- **Flujos (streams):** para datos continuos (audio, video) se establece un **flujo** en vez de mensajes discretos.



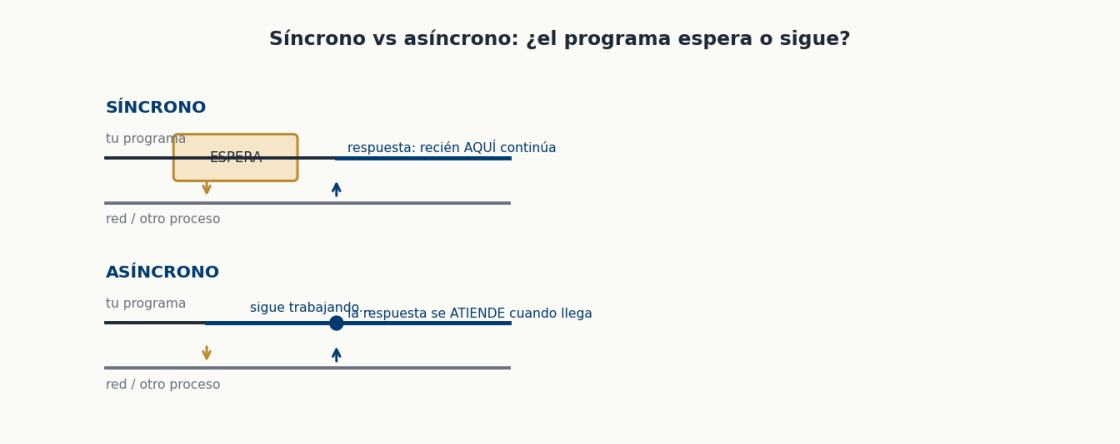

*Modelos de comunicación*

*Figura 3. Paso de mensajes: los datos viajan (sirve entre máquinas). Memoria compartida: una zona común (solo misma máquina).*

> **Idea clave:** en red, la comunicación es por paso de mensajes. Enviarlos y recibirlos bien es el corazón de la Unidad II.

## Tema 4. Síncrono y asíncrono: ¿esperar o seguir?

- **Síncrono (synchronous):** quien ejecuta la operación **queda detenido** hasta que termina. Como una llamada telefónica: no haces otra cosa mientras tanto. Nuestro `recv()` es síncrono.
- **Asíncrono (asynchronous):** la operación se **inicia y el programa continúa**; el resultado se atiende después. Como enviar un mensaje de texto: lo mandas y sigues con tu vida.



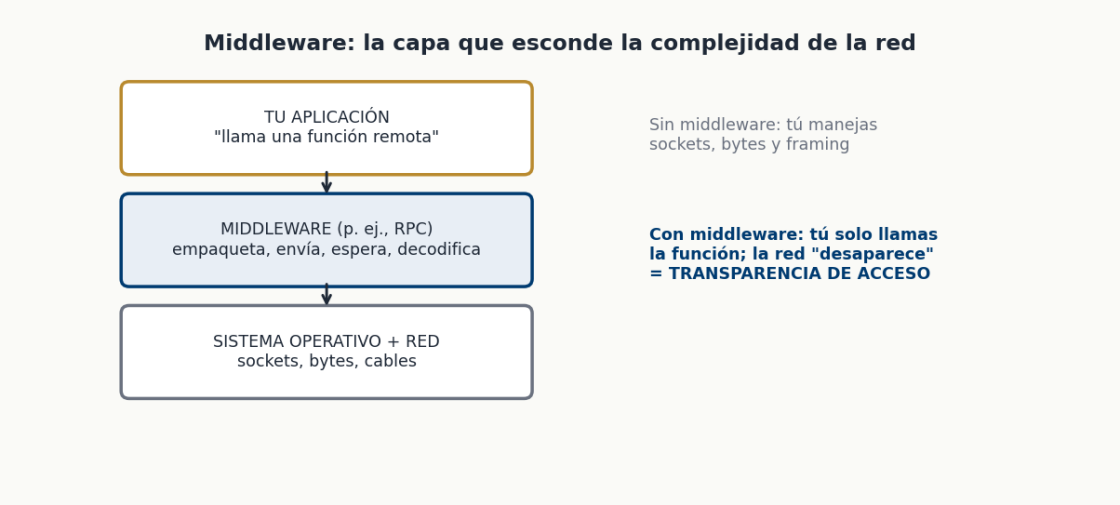

*Síncrono vs asíncrono*

*Figura 4. Síncrono: el programa se detiene hasta la respuesta. Asíncrono: sigue trabajando y la atiende cuando llega.*

> **Idea clave:** síncrono = simple pero te detiene; asíncrono = eficiente pero más complejo. Empezamos síncronos.

## Tema 5. Middleware y transparencia

- **Middleware (software intermedio):** capa de software entre tu aplicación y los detalles de la red/sistema operativo, que **simplifica** construir sistemas distribuidos. Ejemplos del curso: los mecanismos RPC (Unidad III) y los frameworks web (Unidades IV–V).
- **Transparencia (transparency):** que la distribución sea **invisible**. La clave para nosotros: **transparencia de acceso (access transparency)** — usar un recurso **remoto** igual que uno **local**. Otros tipos: de ubicación, de fallos, de replicación.




*Middleware*

*Figura 5. El middleware se interpone entre tu aplicación y la red; cuando lo logra del todo, usar lo remoto se siente local: transparencia de acceso.*

> **Idea clave:** sockets = hablar con la red tú mismo; middleware = alguien habla con la red por ti. Primero aprendemos sockets para entender qué esconde el middleware.

## Tema 6. Protocolos: las reglas del juego

**¿Qué es?** Un **protocolo de red (network protocol)** es el **conjunto de reglas** que dos programas acuerdan para entenderse (Kurose & Ross, cap. 1). Define tres cosas:

1. **Sintaxis:** el **formato** de los mensajes (campos, orden, delimitadores).
2. **Semántica:** el **significado** de cada mensaje.
3. **Temporización (timing): cuándo** se envía cada cosa (quién habla primero, quién responde).

**¿Por qué existe?** Porque los bytes por sí solos no significan nada: si yo envío `ATAQUE A1`, tú solo lo entiendes si acordamos de antemano que ese mensaje existe y qué significa.

**Ejemplo — protocolo de texto de la clase:** `apodo>mensaje`. Sintaxis: apodo + > + mensaje, uno por línea. Semántica: quién habla y qué dice. Temporización: por turnos.

> **Idea clave:** sin protocolo no hay comunicación, solo ruido. Diseñar el protocolo es diseñar la conversación.

## Tema 7. La naturaleza de los datos: bits y bytes

**¿Qué es?** En una computadora —y por tanto en la red— **todo es números**. La unidad mínima es el **bit (binary digit, dígito binario):** un 0 o un 1. Con 8 bits se forma un **byte**, que puede representar $2^8$ = **256 valores distintos** (0 a 255). Un carácter, un píxel, una muestra de audio: todo termina siendo bytes.

**¿Por qué importa en redes?** Porque la red **no sabe** si los bytes que transporta son texto, una foto o una canción: para la red todo es una corriente de bytes. Convertir tus datos a bytes (y reconstruirlos al llegar) es **tu trabajo como programador** — de ahí la regla de oro.

**Unidades y equivalencias:**

| Unidad | Símbolo | Equivalencia | En bytes |
|---|---|---|---|
| bit | b | 0 o 1 | 1/8 de byte |
| byte | B | 8 bits | 1 |
| kilobyte | KB | 1024 B | 1,024 |
| megabyte | MB | 1024 KB | 1,048,576 |
| gigabyte | GB | 1024 MB | 1,073,741,824 |
| terabyte | TB | 1024 GB | 1,099,511,627,776 |

**Referencia rápida de tamaños:** un mensaje de chat ≈ decenas de bytes · una foto de celular ≈ 2–5 MB · una canción ≈ 3–8 MB · un minuto de video HD ≈ 60–100 MB · una película ≈ 1–4 GB.

**Ojo con las velocidades de red:** se miden en **bits** por segundo (Mbps), no en bytes. Una conexión de 100 Mbps transfiere como máximo ≈ 12.5 **MB** por segundo (100 ÷ 8). Por eso una descarga de 1 GB tarda más de un minuto aunque el anuncio diga "100 megas".

> **Idea clave:** la red transporta bytes; ni letras, ni imágenes, ni "mensajes". Todo lo que quieras enviar debe convertirse primero a bytes.

## Tema 8. Codificar y decodificar: ASCII, UTF-8 y la regla de oro

**¿Qué es codificar? Codificar (encode)** es convertir datos a una representación en bytes según una **tabla acordada**; **decodificar (decode)** es el proceso inverso. La tabla más antigua y famosa es **ASCII (American Standard Code for Information Interchange, 1963):** asigna a cada carácter un número de 0 a 127 (un byte le sobra).

**Caracteres ASCII más conocidos (apréndete estos):**

| Decimal | Carácter | Comentario |
|---|---|---|
| 9 | TAB (tabulador) | control: sangría |
| 10 | `\n` (LF, line feed) | **control: salto de línea** |
| 13 | `\r` (CR, carriage return) | control: retorno de carro |
| 32 | (espacio) | el primer carácter "imprimible" |
| 48 | `0` | los dígitos `0`–`9` son 48–57 |
| 65 | `A` | las mayúsculas `A`–`Z` son 65–90 |
| 97 | `a` | las minúsculas `a`–`z` son 97–122 |

**¿Por qué `print("hola\nmundo")` salta de línea?** Porque `\n` no son dos caracteres: es la forma de escribir en el código **el byte número 10** (line feed, "avanzar una línea"), heredado de las teletipos. Cuando en nuestro protocolo decimos "cada mensaje termina con `\n`", estamos diciendo: *cada mensaje termina con el byte 10* — y por eso el receptor puede cortar ahí. Es un **delimitador** (lo usaremos en el Tema 15 para el framing).

**El límite de ASCII:** solo 128 caracteres — no caben `ñ`, `á`, `é`, ü ni emojis. La solución mundial es **UTF-8**: una codificación que **extiende ASCII** (los primeros 128 caracteres son idénticos) y usa **de 1 a 4 bytes por carácter** para cubrir todos los idiomas y símbolos. Por eso `"ñ".encode("utf-8")` produce **2 bytes**, no 1.

**En Python (memoriza estas dos líneas):**

```
datos = mensaje.encode("utf-8")   # texto  -> bytes   (para ENVIAR)
texto = datos.decode("utf-8")     # bytes  -> texto   (al RECIBIR)
```

- `send()` / `sendto()` **exigen bytes**: enviar texto sin codificar lanza `TypeError: a bytes-like object is required`.
- `recv()` / `recvfrom()` **devuelven bytes**: impresos crudos se ven como `b'hola'` (la `b` delata que son bytes).

**Idea clave:** codificar = texto → bytes con una tabla acordada (UTF-8). Si ambos extremos usan la misma tabla, cualquier texto —con acentos y emojis— viaja sin perderse.

## Tema 9. Direcciones IP, puertos y estándares

**Dirección IP (IP address):** identifica una **máquina** en la red. IPv4 = 4 números de 0 a 255 (`192.168.1.50`). Caso especial: `127.0.0.1` / `localhost` = **esta misma computadora** (para practicar sin red).

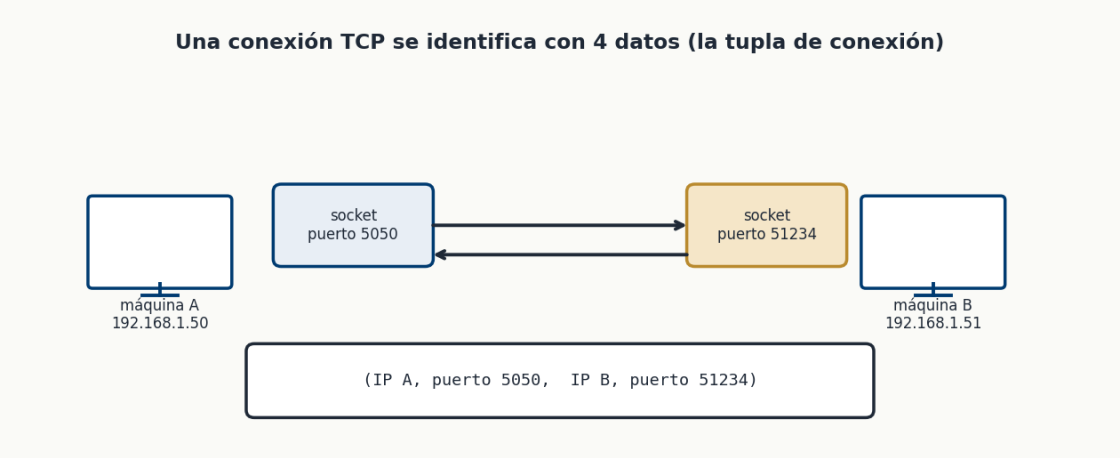
**Puerto (port):** número de **16 bits** (0–65535) que identifica **qué programa** dentro de la máquina recibe los datos. La **IANA** (el organismo que estandariza estos números) los divide en tres rangos:

- **Bien conocidos (well-known), 0–1023:** reservados a servicios estándar (tabla abajo).
- **Registrados (registered), 1024–49151:** asignados a aplicaciones específicas.
- **Dinámicos/privados (efímeros), 49152–65535:** los que el sistema operativo asigna automáticamente a los clientes.

**Puertos bien conocidos que debes reconocer:**

| Puerto | Protocolo / Servicio | ¿Qué hace? |
|---|---|---|
| 20–21 | FTP | Transferencia de archivos (21 control, 20 datos) |
| 22 | SSH | Acceso remoto cifrado a una terminal |
| 23 | Telnet | Acceso remoto en texto plano (histórico, inseguro) |
| 25 | SMTP | Envío de correo electrónico entre servidores |
| 53 | DNS | Traduce nombres (ej. `uv.mx`) a direcciones IP; usa UDP |
| 67–68 | DHCP | Asigna IP automáticamente a las máquinas de una red |
| 80 | HTTP | Páginas web sin cifrar |
| 110 | POP3 | Descarga de correo al cliente |
| 143 | IMAP | Lectura de correo dejándolo en el servidor |
| 443 | HTTPS | Web cifrada (HTTP sobre TLS) |
| 3306 | MySQL | Base de datos MySQL/MariaDB |
| 3389 | RDP | Escritorio remoto de Windows |
| 5050 | (el de nuestras prácticas) | Puerto alto libre que usamos en clase |

Una conexión TCP queda **unívocamente identificada** por la **tupla de conexión**: `(IP local, puerto local, IP remota, puerto remoto)`.




*La tupla de conexión*

*Figura 6. La conexión se identifica con 4 datos; por eso muchos clientes pueden hablar con el mismo servidor en el mismo puerto: cada tupla es distinta.*

> **Idea clave:** IP = qué máquina; puerto = qué programa; la tupla de 4 = qué conversación. Los puertos 0–1023 están estandarizados: el 80 siempre será HTTP.

---

## Tema 10. TCP: qué es, para qué sirve y cómo funciona

**¿Qué es? TCP (Transmission Control Protocol):** el protocolo de transporte **orientado a conexión** y **confiable** (Kurose & Ross, cap. 3; Rhodes & Goerzen, 2014, cap. 3).

**¿Para qué sirve?** Para aplicaciones donde **no se puede perder nada**: páginas web, correo, transferencia de archivos, inicios de sesión, operaciones bancarias.

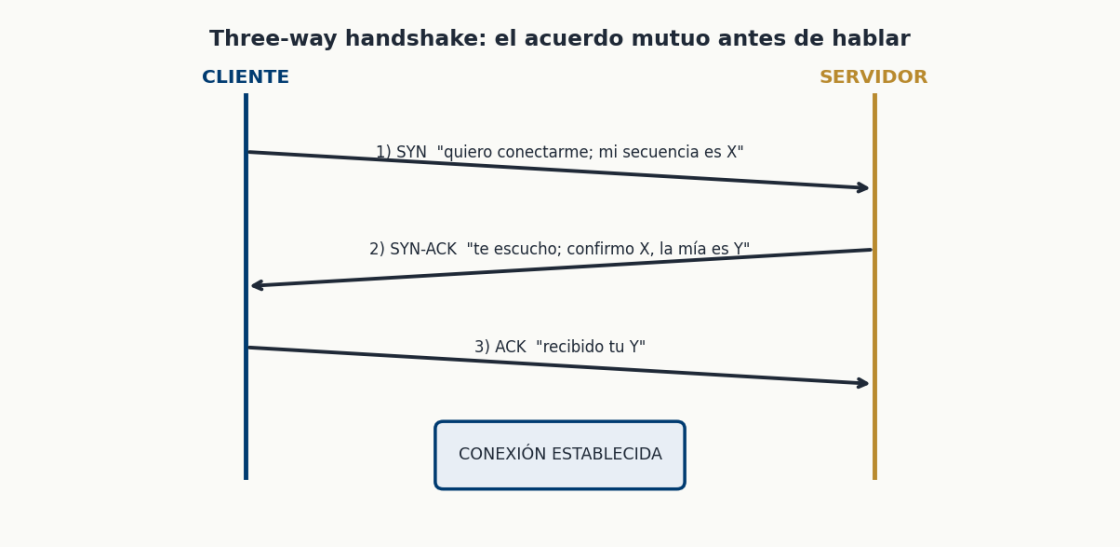
**¿Qué garantiza?**

- **Conexión previa:** antes de los datos hay un acuerdo formal (el handshake).
- **Entrega garantizada:** cada byte es confirmado (**ACK, acknowledgement**); si no hay confirmación, TCP **retransmite** solo.
- **Orden garantizado:** los bytes llegan en el orden en que se enviaron.
- **Control de flujo y congestión:** regula la velocidad para no ahogar al receptor ni a la red.

**¿Qué NO garantiza?** Fronteras de mensaje: TCP entrega un **flujo continuo de bytes (byte stream)**, no "mensajes" (ese es el problema del framing, Tema 15).

### El three-way handshake (apretón de manos de tres vías)

Cuando el cliente llama `connect()`, TCP ejecuta **bajo el agua** un intercambio de tres mensajes:




*Three-way handshake*

*Figura 7. SYN, SYN-ACK, ACK: ambos extremos comprueban que el otro existe, escucha y responde, antes de enviar datos.*

1. **SYN:** el cliente dice "quiero conectarme; mi número de secuencia inicial es X".
2. **SYN-ACK:** el servidor responde "te escucho; confirmo tu X; la mía es Y".
3. **ACK:** el cliente confirma "recibido tu Y".

Solo entonces la conexión queda **ESTABLISHED** y el `accept()` del servidor despierta. ¿Por qué tres y no dos? Porque **ambos** lados deben confirmar que el otro existe y puede responder: es un **acuerdo mutuo**.

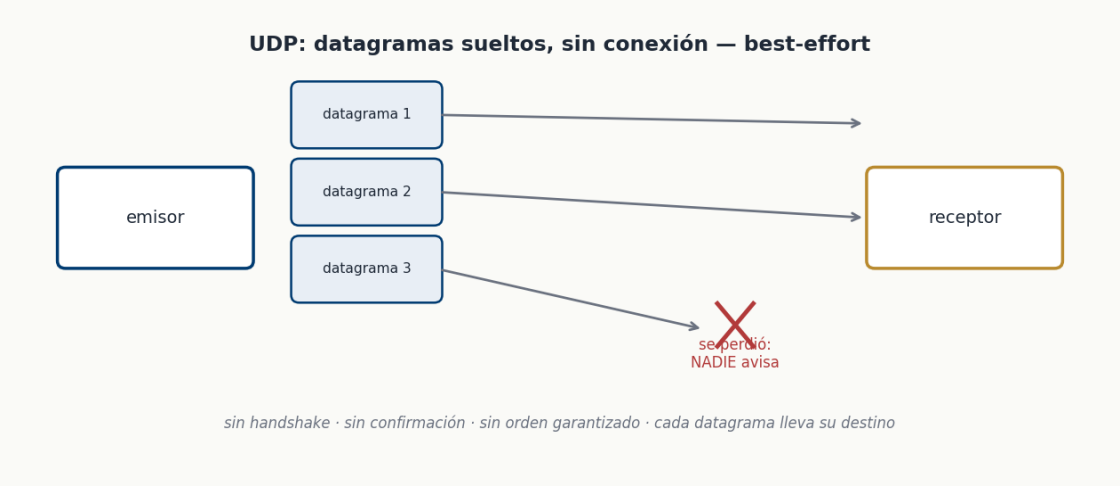
> **Idea clave:** TCP = confianza mediante acuerdo y confirmación. Todo eso cuesta tiempo: esa es la cuota que UDP no paga.

## Tema 11. UDP: qué es, para qué sirve y cómo funciona

**¿Qué es? UDP (User Datagram Protocol):** el protocolo de transporte **sin conexión (connectionless)** y **best-effort (de mejor esfuerzo)** (Kurose & Ross, cap. 3; Rhodes & Goerzen, 2014, cap. 2).

**¿Para qué sirve?** Para aplicaciones donde la **velocidad manda** y **perder un dato ocasional se tolera**: streaming de video, voz sobre IP, juegos en línea, consultas DNS (puerto 53). Lógica: retransmitir un cuadro de video perdido llegaría demasiado tarde para servir.

**¿Cómo funciona?** Los mensajes viajan como **datagramas (datagrams):** paquetes **independientes**, cada uno con su destino escrito, como cartas echadas al buzón. No hay handshake, no hay confirmación, no hay retransmisión, no hay orden garantizado. Si un datagrama se pierde, **nadie avisa** — ni al emisor ni al receptor.




*UDP: datagramas sin conexión*

*Figura 8. UDP lanza datagramas independientes; si uno se pierde, nadie avisa. A cambio: velocidad y ligereza.*

**Precisión importante (Kurose & Ross, cap. 3):** UDP **no** es para aplicaciones que no pueden permitir pérdida de datos — es exactamente al contrario. UDP se elige para aplicaciones que **toleran** pérdida a cambio de velocidad; quien garantiza la entrega es TCP.

### TCP vs UDP: cuándo usar cada uno

| Característica | TCP | UDP |
|---|---|---|
| Conexión | Sí (handshake SYN/SYN-ACK/ACK) | No |
| Entrega | **Garantizada** (acuses + retransmisión) | No garantizada (best-effort) |
| Orden | Garantizado | No garantizado |
| Unidad de datos | Flujo de bytes (requiere framing) | Datagramas (cada envío es un mensaje) |
| Si algo falla | Te enteras | Nadie avisa |
| Velocidad / sobrecarga | Menor / mayor | Mayor / mínima |
| Para qué sirve | Web, correo, archivos, banca | Streaming, VoIP, juegos, DNS |

> **Idea clave:** TCP y UDP no son "bueno y malo": son **herramientas para distinto problema**. Elegir mal (UDP para nómina, TCP para video en vivo) es un error de diseño.

## Tema 12. Cómo viajan los datos grandes: archivos, video y tiempo real

**¿Por qué no viaja todo "de una vez"?** La red transporta datos en **paquetes (packets)** de tamaño limitado (típicamente ~1500 bytes por paquete en Ethernet: la **MTU, Maximum Transmission Unit**). Una foto de 3 MB son **miles de paquetes**. Además, tu programa no debería intentar mandar todo de golpe: se envía **por fragmentos (chunks)**.

**Enviar un archivo (foto, video, audio) por TCP:** el patrón es leer el archivo en pedazos y enviarlos en ciclo:

```
with open("foto.jpg", "rb") as archivo:      # "rb" = leer como BYTES
    while True:
        fragmento = archivo.read(4096)        # lee un pedazo de 4 KB
        if not fragmento:                     # se acabo el archivo
            break
        conexion.sendall(fragmento)           # envia el pedazo completo
```

El receptor acumula los fragmentos y los escribe en disco en el mismo orden. TCP garantiza que lleguen **todos, completos y en orden**; si un paquete se pierde, se retransmite sin que tu programa se entere. (Nota fina: `sendall()` insiste hasta que el fragmento salió completo; y para saber cuándo terminó el archivo se usa el framing del Tema 15: prefijo de longitud o cerrar la conexión como marca de fin.)

**Video en tiempo real (streaming):** aquí el razonamiento es opuesto. Lo que importa es que los cuadros lleguen **ya**: si un cuadro se pierde, retransmitirlo llegaría cuando ya pasó de moda. Por eso el streaming usa **UDP**: un flujo continuo de datagramas, y el reproductor guarda un **búfer (buffer)** de unos segundos para suavizar huecos; si falta un cuadro, se omite y se sigue.




*Datos grandes y streaming*

*Figura 9. Archivos: por fragmentos, TCP recupera lo perdido. Tiempo real: flujo continuo por UDP, lo perdido se deja ir.*

> **Idea clave:** todo viaja dividido en paquetes pequeños. La diferencia es qué pasa con los que se pierden: TCP los recupera (archivos); el streaming los deja ir (tiempo real).

## Tema 13. ¿Qué es un socket?

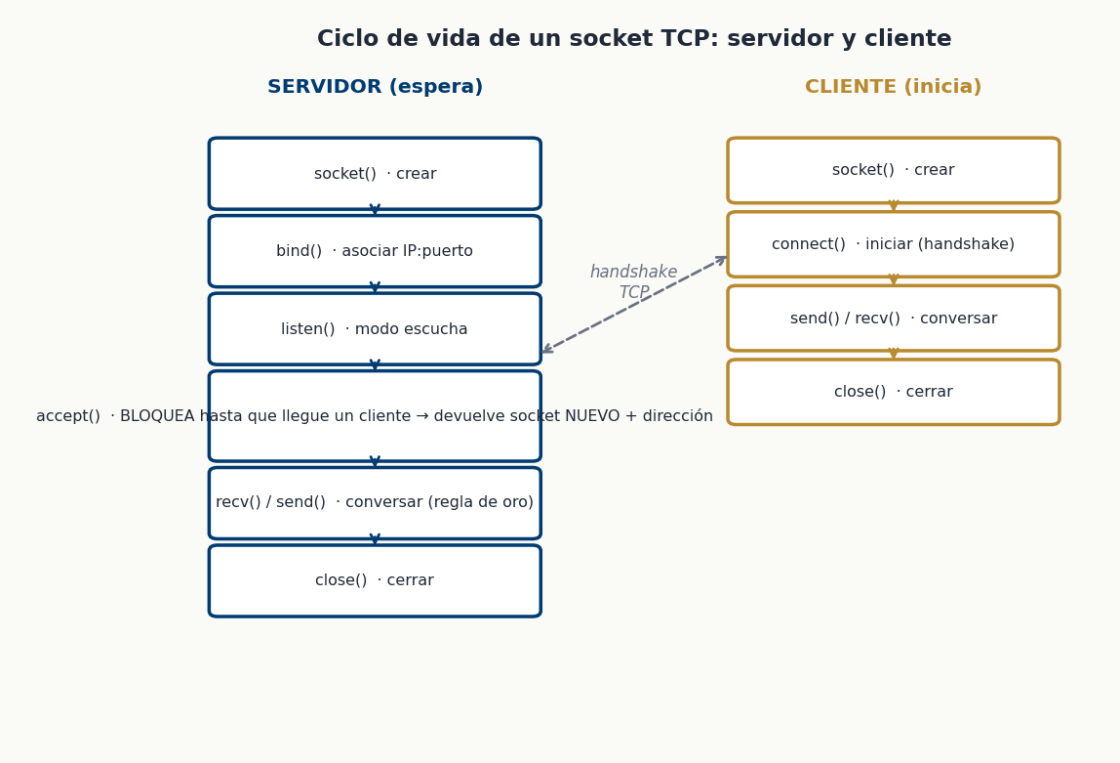
**¿Qué es?** Un **socket (enchufe)** es el **punto final de una comunicación**: la **interfaz (API)** que el sistema operativo le da a tu programa para enviar y recibir datos por la red. Es la API de **Berkeley sockets**, presente casi idéntica en todos los lenguajes (Rhodes & Goerzen, 2014, cap. 1; Stevens et al., 2004).

**Analogía:** un teléfono. Para hablar necesitas tener aparato (`socket()`), tener número propio (`bind()`), dejarlo disponible (`listen()`), contestar cuando llaman (`accept()`), o marcar tú (`connect()`); luego hablar/escuchar (`send()`/`recv()`) y colgar (`close()`).

```python
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)    # socket TCP sobre IPv4
```

- `AF_INET` = familia de direcciones **IPv4**.
- `SOCK_STREAM` = flujo de bytes confiable → **TCP**.
- `SOCK_DGRAM` = datagramas → **UDP**.

## Tema 14. Ciclo de vida del socket TCP




*Ciclo de vida TCP*

*Figura 10. El ciclo completo: el servidor prepara y espera; el cliente inicia; el handshake los enlaza; luego conversan.*

### Lado servidor — cada llamada, qué hace y por qué existe

**1. `socket(AF_INET, SOCK_STREAM)` — crear.** Pide al sistema operativo el punto final. Todavía no tiene dirección ni conexión: es un teléfono sin número.

**2. `bind((host, puerto))` — asociar.** Le da al socket su **dirección local**: en qué IP y puerto de ESTA máquina atiende. *¿Por qué existe?* Porque el servidor debe estar en una dirección **conocida y fija** para que los clientes lo encuentren. `bind(("0.0.0.0", 5050))` = "en todas las interfaces de esta máquina, puerto 5050". Si el puerto está ocupado: `OSError: Address already in use`; la buena práctica:

```python
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

**3. `listen(backlog)` — escuchar.** Convierte el socket en **socket de escucha**. El argumento es el **backlog**: cuántas conexiones pueden quedar **en espera** mientras atendemos otra (no es el máximo de clientes total).

**4. `accept()` — aceptar. ← el paso clave del examen. Se bloquea** hasta que un cliente se conecta, y devuelve **DOS cosas**:

```
conexion, direccion = servidor.accept()
```

- `conexion`: un **socket NUEVO**, dedicado exclusivamente a hablar con ESE cliente.
- `direccion`: la tupla `(IP, puerto)` del cliente.

*¿Por qué un socket nuevo?* Porque el socket original tiene otro trabajo: seguir siendo el **recepcionista** que acepta futuros clientes. La conversación ocurre por el socket nuevo. **Nunca** envíes ni recibas por el socket de escucha.

**5. `recv(n)` / `send(bytes)` — conversar.** `recv(1024)` recibe **hasta** 1024 bytes (bloqueante; si el cliente cerró, devuelve `b''` vacío — así se detecta el cierre). Regla de oro: encode al enviar, decode al recibir.

**6. `close()` — cerrar.** Se cierran **ambos** sockets: primero `conexion`, luego `servidor`.

### Lado cliente

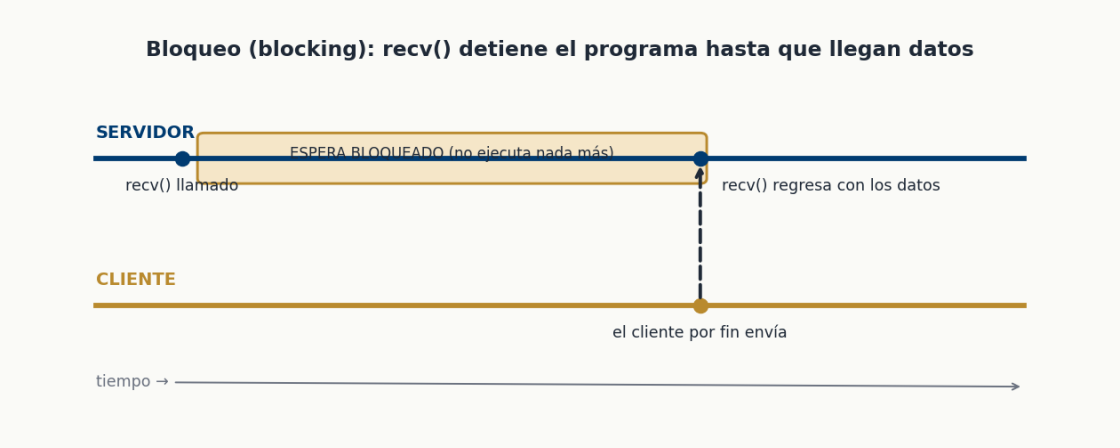
**1. `socket()`** — igual. **2. `connect((ip, puerto))`** — **inicia** la conexión; internamente dispara el **three-way handshake**; si nadie escucha: `ConnectionRefusedError`. El cliente **no hace `bind()`**: el sistema operativo le asigna un **puerto efímero** automático. **3. `send()` / `recv()`** — conversar. **4. `close()`** — colgar.

> **Idea clave:** servidor = crear, **asociar, escuchar, aceptar**, conversar, cerrar. Cliente = crear, **conectar**, conversar, cerrar. Las diferencias son exactamente las del modelo cliente-servidor: uno espera, otro inicia.

## Tema 15. Bloqueo y framing: los dos detalles que más caen en examen

### Bloqueo (blocking)

Por defecto, las llamadas de socket son **bloqueantes**: la ejecución se **detiene** hasta que el evento ocurre. `accept()` espera un cliente; `connect()` espera el handshake; `recv()` espera datos.




*Línea de tiempo del bloqueo*

*Figura 11. `recv()` detiene al servidor mientras el cliente no envía nada. No es un fallo: es el comportamiento por defecto.*

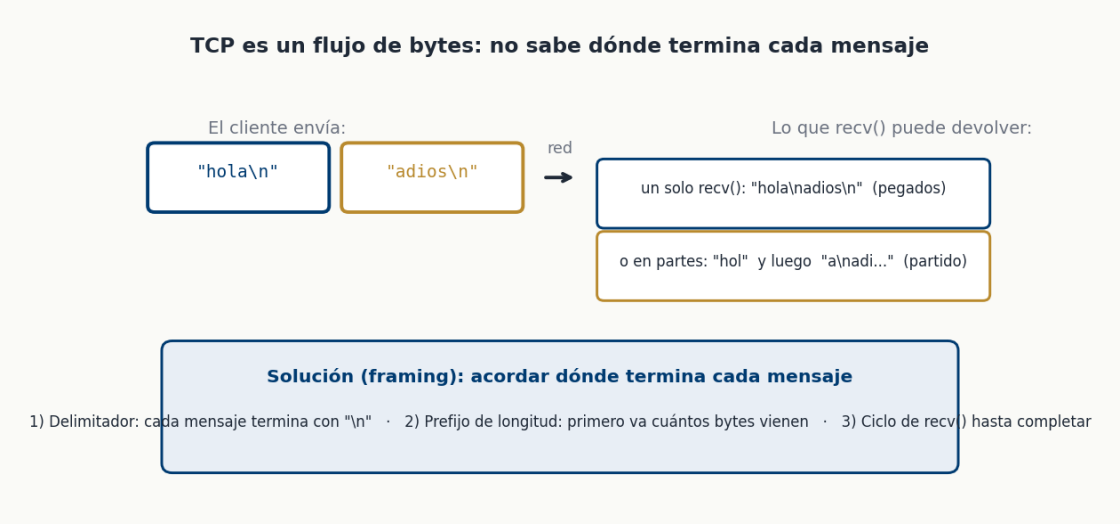
**Demostración reproducible** (el cliente se conecta y calla 4 segundos; la puedes correr con el Código 3):

```
servidor> voy a llamar recv() ... (aqui me BLOQUEO hasta que llegue algo)
servidor> recv() regreso despues de 4.0 segundos de espera
```

Esto explica el síntoma clásico “mi programa se congeló”: casi siempre es un `recv()` (o `accept()`) esperando algo que no llega. **Salidas:** `settimeout(segundos)` (espera con límite; al expirar lanza `socket.timeout`) y, más adelante, concurrencia (un hilo por cliente).

### Framing (encuadre de mensajes)

TCP **no transporta “mensajes”: transporta un flujo de bytes.** Entonces `recv(1024)` NO significa “dame un mensaje” sino “dame **hasta** 1024 bytes de lo que haya llegado”: puede traer un mensaje **partido** o varios **pegados**.




*El problema del framing y sus soluciones*

*Figura 12. Los mensajes pueden llegar pegados o partidos. Definir dónde termina cada mensaje (framing) es responsabilidad de TU protocolo.*

**Las tres soluciones clásicas** (Rhodes & Goerzen, 2014, cap. 5):

1. **Delimitador (delimiter):** cada mensaje termina con una marca acordada —normalmente el byte 10, `\n` (Tema 8)— y el receptor acumula hasta encontrarla. *(La que usamos en clase.)*
2. **Prefijo de longitud (length prefix):** primero se envía el tamaño (p. ej., 4 bytes con el número de bytes), luego el mensaje; el receptor lee exactamente esa cantidad.
3. **Ciclo de recepción (receive loop):** llamar `recv()` **repetidamente** hasta completar el mensaje según la regla acordada (1 y 2 siempre se combinan con 3).

## Tema 16. El ciclo de vida en UDP

Más corto, porque **no hay conexión**:

- **Servidor UDP:** `socket(SOCK_DGRAM) → bind() → recvfrom() / sendto() → close()`. **No hay `listen()` ni `accept()`.**
- **Cliente UDP:** `socket(SOCK_DGRAM) → sendto() / recvfrom() → close()`. **No hay `connect()`.**

Diferencias finas de la API:

- `recvfrom(n)` devuelve **`(datos, direccion)`**: los datos **y quién los envió** (sin conexión, es la única forma de saberlo y de poder responder).
- `sendto(bytes, (ip, puerto))` exige el **destino en cada envío**.

## Tema 17. Errores típicos y qué significan

| Error / síntoma | Causa | Solución |
|---|---|---|
| `OSError: Address already in use` | El puerto sigue ocupado por una corrida anterior | `SO_REUSEADDR` antes de `bind()`; o cambiar de puerto |
| `ConnectionRefusedError` | `connect()` sin servidor escuchando | Arrancar primero el servidor; verificar IP/puerto |
| "Se congeló" | Bloqueado en `recv()/accept()` | Es lo normal; verificar que el otro lado envíe; usar `settimeout()` |
| Imprime `b'hola'` | Bytes sin decodificar | `.decode("utf-8")` al recibir |
| `TypeError: a bytes-like object is required` | Texto sin codificar al enviar | `.encode("utf-8")` al enviar |
| Mensajes pegados o partidos | Falta de framing | Delimitador `\n` / prefijo de longitud + ciclo de recepción |
| `listen()` en UDP | UDP no tiene conexiones que escuchar | Quitarlo: `socket → bind → recvfrom/sendto → close` |

## Tema 18. RPC: el puente hacia la Unidad III

**¿Qué es? RPC (Remote Procedure Call, llamada a procedimiento remoto):** ejecutar una función que **vive en otra máquina** como si fuera local (López-Fuentes, 2015, sec. 4.5). Es el primer **middleware** del curso; su meta es la **transparencia de acceso**.

**¿Por qué existe?** Porque programar con sockets crudos es trabajoso (bytes, framing, protocolos). RPC esconde todo eso bajo una simple llamada de función.

**Los actores: stub (talón), lado cliente** — función local que *empaqueta* los argumentos, los envía y espera; **skeleton (esqueleto), lado servidor** — recibe, *desempaqueta*, ejecuta la función real y devuelve el resultado. Empaquetar/desempaquetar se llama **marshalling / unmarshalling**.

**Los 10 pasos:** (1) el cliente llama al stub → (2) el stub empaca los parámetros → (3) se envían por la red → (4) el SO del servidor los recibe → (5) el skeleton desempaca → (6) ejecuta el procedimiento real → (7) empaca el resultado → (8) viaja de regreso → (9) el stub desempaca → (10) el cliente recibe el valor **como si la función fuera local**.

> **Idea clave:** RPC no reemplaza a los sockets — está **construido encima** de ellos. Por eso primero aprendemos sockets: para entender qué esconde el middleware.

## Tema 19. Repaso de Python: lo mínimo antes de programar en red: lo mínimo para programar en red

Todo el curso se programa en **Python 3**. Este tema es el cimiento: si algo de aquí no te queda claro, los códigos de red se van a sentir como magia. No lo son: todo es lo siguiente.

### 0.1 Instalar y verificar Python

- **Descarga oficial:** https://www.python.org/downloads/ (elige Python 3.12 o superior; en Windows marca la casilla **"Add Python to PATH"** durante la instalación).
- **Verificar la instalación** (en una terminal):

```
python --version
```

debe responder algo como `Python 3.12.x` (si no, prueba `python3 --version`).

- **Editor recomendado del curso:** VS Code (https://code.visualstudio.com/), pero cualquier editor sirve.
- **Cómo se corren los programas del curso:** en **dos terminales** — primero arranca el **servidor** (`python eco_tcp_servidor.py`), luego el **cliente** (`python eco_tcp_cliente.py`). El servidor debe estar vivo ANTES de que el cliente intente conectar.

## 0.2 Variables

Una **variable (variable)** es un **nombre que guarda un valor** en memoria. En Python no se declara el tipo: el valor lo define (tipado dinámico).

```
PUERTO = 5050                # int (entero): un numero sin decimales
IP_SERVIDOR = "localhost"    # str (cadena): texto entre comillas
ACTIVO = True                # bool (booleano): True o False
datos = b"hola"              # bytes: una secuencia de bytes (ojo con la b)
```

Los cuatro tipos aparecen **literalmente** en todos los programas del curso. El que más importa en redes es `bytes`: es lo único que viaja (Tema 7).

## 0.3 Operadores

- **Aritméticos:** `+` `-` `*` `/` (división con decimales) · `//` (división entera) · `%` (residuo).
- **Comparación (devuelven bool):** `==` igual · `!=` distinto · `<` `>` `<=` `>=`. Ojo: `=` **asigna**, `==` **compara** — confundirlos es un error clásico.
- **Lógicos:** `and`, `or`, `not`.
- **Pertenencia:** `in` — `"a" in "hola"` es `True`. Lo usamos en el framing: `while "\n" in buffer:` (¿ya llegó un mensaje completo?).

## 0.4 Estructuras de datos básicas

| Estructura | ¿Qué es? | Mutable | Ejemplo en el curso |
|---|---|---|---|
| `str` (cadena) | texto, secuencia de caracteres | No | `mensaje = "hola"` |
| `bytes` | secuencia de bytes (lo que viaja) | No | `datos = b"hola"` |
| `list` (lista) | colección **ordenada y modificable** | Sí | `mensajes = ["hola", "adios"]` |
| `tuple` (tupla) | colección **ordenada e inmodificable** | No | `("192.168.1.50", 5050)` — la dirección de `accept()` |
| `dict` (diccionario) | pares **clave: valor** | Sí | `{"apodo": "ana", "texto": "hola"}` |
| `set` (conjunto) | elementos **únicos, sin orden** | Sí | la flota del juego: coordenadas sin repetir |

Fíjate en la tupla: cuando `accept()` devuelve `(conexion, direccion)` o `recvfrom()` devuelve `(datos, direccion)`, está devolviendo **tuplas** — por eso se "desempacan" así:

```
conexion, direccion = servidor.accept()   # desempacar (unpack) una tupla de 2
```

## 0.5 Funciones

Una **función (function)** es un bloque de código **con nombre** que puedes reutilizar: recibe **parámetros (parameters)** y puede **devolver (return)** un resultado.

```python
def armar_eco(mensaje):          # def + nombre + (parametros)
    return "ECO> " + mensaje     # return: lo que la funcion entrega

respuesta = armar_eco("hola")    # llamada: respuesta vale "ECO> hola"
```

**¿Para qué?** Para no repetir código: si escribes el mismo bloque dos veces, va en una función (norma de código limpio, 0.9).

## 0.6 El ciclo `while True` (el corazón de los servidores)

`while` repite un bloque **mientras** su condición sea verdadera. `while True:` es **siempre verdadero**: un **ciclo infinito** — y la única salida es la instrucción `break`.

¿Por qué los servidores lo usan? Porque un servidor **no sabe cuántos mensajes va a recibir**: debe atender *indefinidamente*, hasta que pase algo (el cliente cerró, o llegó un mensaje de salida):

```python
while True:
    datos = conexion.recv(1024)   # espera un mensaje
    if not datos:                 # condicion de salida: el cliente cerro
        break                     # rompe el ciclo infinito
    print("Me llego:", datos.decode())
```

> **Idea clave:** `while True` + `break` = "hazlo una y otra vez hasta que yo diga". Sin el `break`, el programa nunca termina (a veces es justo lo que quieres en un servidor; a veces es un bug).

## 0.7 Librerías: código que otros ya escribieron por ti

Una **librería (library)** o **módulo (module)** es código hecho por otros que **reusas** en tu programa con `import`. Python trae una **librería estándar (standard library)** gigante ya instalada — `socket`, `time`, `json`, `math`— y miles de librerías externas instalables.

```python
import socket          # trae el modulo socket a tu programa
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)   # usas lo que contiene
```

Todo el curso funciona con `socket` y `time` de la librería estándar: **no hay que instalar nada** para las prácticas de las Unidades I y II.

## 0.8 Entornos virtuales (venv): una isla por proyecto

Un **entorno virtual (virtual environment, venv)** es una **carpeta aislada** con su propia copia de Python y sus propias librerías, independiente del resto de la computadora.

**¿Para qué?** Para que cada proyecto tenga sus versiones sin pelear con otros proyectos (el proyecto A puede usar una versión de una librería y el B otra, sin romperse mutuamente).

```
python -m venv .venv                # crear el entorno (carpeta .venv)
source .venv/bin/activate           # activarlo (Linux/Mac)
.venv\Scripts\activate              # activarlo (Windows)
pip install <libreria>              # instalar algo DENTRO del entorno
deactivate                          # salir del entorno
```

En este curso lo usaremos cuando lleguen las librerías externas (Unidades IV–V, p. ej., Flask).

## 0.9 Código limpio (clean code): las normas del curso

El código se escribe **para que lo lean humanos**, no solo para que corra la máquina. Normas mínimas que se evalúan en todas las prácticas:

- **Nombres entendibles:** `conexion`, `mensaje`, `direccion_cliente`. Prohibido `x`, `c`, `d`, `var1`.
- **Constantes en MAYÚSCULAS** al inicio: `PUERTO = 5050`. Nada de números "mágicos" sueltos.
- **Comentarios que explican el porqué**, no lo obvio: `# accept() BLOQUEA hasta que llegue un cliente` sí; `# aqui hago recv` no.
- **Un solo idioma** en código y comentarios (español; los comandos de Python van en inglés porque así es el lenguaje).
- **Sin código muerto:** borra líneas comentadas, `print` de prueba y variables sin uso.
- **Sin repetición:** si copiaste un bloque dos veces, conviértelo en función.
- **Indentación consistente:** 4 espacios, sin mezclar tabuladores.

**Antes / después:**

```
# ANTES (no se vale)
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.bind(('',9000))
s.listen(1)
c,a=s.accept()
d=c.recv(1024)
print(d)

# DESPUES (asi si)
PUERTO = 9000

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("localhost", PUERTO))
servidor.listen(1)

# accept() devuelve un socket NUEVO para hablar con este cliente
conexion, direccion = servidor.accept()
datos = conexion.recv(1024)
print("Me llego:", datos.decode())
```

---

---

## Tema 20. Códigos completos y comentados

Todos fueron **ejecutados y verificados**. Léelos comentario por comentario: son la referencia de "código bien escrito" del curso (constantes en mayúsculas, nombres entendibles, comentarios que explican el *porqué*). Para practicar en una sola computadora usa `"localhost"` como IP.

### Código 1 — Eco TCP: servidor

```
#!/usr/bin/env python3
# ============================================================
#  TCP ECHO SERVER (SERVIDOR ECO TCP)
#  Programacion en Red · Universidad Veracruzana
# ------------------------------------------------------------
#  Que hace: repite (echo = eco) todo lo que el cliente mande.
#  Ciclo de vida del SERVER (servidor):
#    socket() -> bind() -> listen() -> accept() -> recv/send -> close()
#  Regla de oro: ENVIAR = ENCODE · RECIBIR = DECODE
```

```
# ============================================================

import socket

PUERTO = 5050   # port (puerto) acordado entre cliente y servidor

# 1) socket(): crear el stream socket (socket de flujo) TCP sobre IPv4
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2) bind(): asociar el socket a (IP local, puerto)
#    "0.0.0.0" = todas las interfaces de red de esta maquina
servidor.bind(("0.0.0.0", PUERTO))

# 3) listen(): pasar a modo escucha; 1 = backlog (cuantas conexiones esperan en fila)
servidor.listen(1)
print("Servidor eco escuchando en el puerto", PUERTO, "...")

# 4) accept(): BLOCKING (bloqueante) -- el programa SE DETIENE aqui
#    hasta que un cliente se conecte. Devuelve DOS cosas:
#    un socket NUEVO para hablar con ese cliente, y su direccion.
conexion, direccion = servidor.accept()
print("Cliente conectado desde", direccion)

while True:
    datos = conexion.recv(1024)     # recv tambien es bloqueante
    if not datos:                   # datos vacios (b'') = el cliente cerro
        break
    mensaje = datos.decode("utf-8") # RECIBIR = DECODE
    print("Me llego:", mensaje)
    eco = "ECO> " + mensaje
    conexion.send(eco.encode("utf-8"))  # ENVIAR = ENCODE

# 5) close(): se cierran AMBOS sockets (la conversacion y el receptor)
conexion.close()
servidor.close()
print("Servidor eco terminado.")
```

**Salida real verificada:** el servidor imprime `Me llego: hola` y el cliente recibe `ECO> hola`.

## Código 2 — Eco TCP: cliente

```
#!/usr/bin/env python3
# ============================================================
#  CLIENTE DEL ECO TCP
#  Ciclo de vida del CLIENT (cliente):
#    socket() -> connect() -> send/recv -> close()
#  El cliente INICIA: no usa bind/listen/accept, sino connect().
# ============================================================

import socket

IP_SERVIDOR = "localhost"   # o la IP de la maquina del servidor, p. ej. "192.168.1.50"
PUERTO = 5050               # debe ser el MISMO puerto del servidor

cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
# connect(): inicia la conexion (dispara el three-way handshake).
# Si nadie escucha en esa IP:puerto, lanza ConnectionRefusedError.
cliente.connect((IP_SERVIDOR, PUERTO))
print("Conectado al servidor eco. Escriba mensajes (vacio para salir).")

while True:
    mensaje = input("Usted dice: ")
    if mensaje == "":
        break
    cliente.send(mensaje.encode("utf-8"))        # ENVIAR = ENCODE
    eco = cliente.recv(1024).decode("utf-8")     # RECIBIR = DECODE
    print("El servidor responde:", eco)

cliente.close()
print("Conexion cerrada.")
```

**Cómo ejecutarlo (dos terminales):**

1. **Terminal 1 (servidor):** `python eco_tcp_servidor.py` → debe mostrar `Servidor eco escuchando en el puerto 5050 ...` y quedarse esperando (está bloqueado en `accept()`).
2. **Terminal 2 (cliente):** `python eco_tcp_cliente.py` → debe mostrar `Conectado al servidor eco...`; en la terminal 1 aparece `Cliente conectado desde (...)`.
3. En la terminal 2 escribe `hola` y Enter → terminal 2 muestra `El servidor responde: ECO> hola` y terminal 1 muestra `Me llego: hola`.
4. Para terminar: deja el mensaje vacío en el cliente; el servidor detecta el cierre (`b''`) y termina solo.

## Código 3 — Demostración del bloqueo

```
#!/usr/bin/env python3
# bloqueo_servidor.py -- demuestra que recv() es BLOCKING (bloqueante)
import socket
import time

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # reusar el puerto
servidor.bind(("localhost", 5199))
servidor.listen(1)
print("servidor> esperando conexion...")

conexion, direccion = servidor.accept()
print("servidor> cliente conectado:", direccion)

print("servidor> voy a llamar recv() ... (aqui me BLOQUEO hasta que llegue algo)")
inicio = time.time()
datos = conexion.recv(1024)          # <-- AQUI OCURRE EL BLOQUEO
espera = time.time() - inicio
print("servidor> recv() regreso despues de {:.1f} segundos de espera".format(espera))
print("servidor> llego:", datos.decode())

conexion.close()
servidor.close()
```

```
#!/usr/bin/env python3
# bloqueo_cliente.py -- se conecta y CALLA unos segundos antes de enviar
```

```python
import socket
import time

ESPERA = 4   # segundos que el cliente permanece callado

cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("localhost", 5199))
print("cliente> conectado; voy a callar", ESPERA, "segundos antes de enviar...")
time.sleep(ESPERA)
cliente.send("ya llegue!".encode())
cliente.close()
```

**Salida real verificada:** `recv() regreso despues de 4.0 segundos de espera` — el servidor quedó bloqueado exactamente lo que el cliente calló.

**Cómo ejecutarlo (dos terminales):**

1. **Terminal 1:** `python bloqueo_servidor.py` → muestra `esperando conexion...`.
2. **Terminal 2:** `python bloqueo_cliente.py` → el cliente avisa que callará 4 segundos.
3. **Observa la terminal 1:** el mensaje `recv() regreso despues de 4.0 segundos...` aparece **solo cuando el cliente envía**. Ese tiempo medido ES el bloqueo.

## Código 4 — Eco UDP: servidor y cliente

```python
#!/usr/bin/env python3
# ============================================================
#  ECO UDP (SERVIDOR)
#  datagram socket (socket de datagramas): SIN listen/accept.
#  recvfrom() devuelve (datos, direccion); sendto() responde ahi.
#  UDP es best-effort (de mejor esfuerzo): si un datagrama se
#  pierde, NADIE se entera -- no hay retransmision.
# ============================================================

import socket

PUERTO = 5050

servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)   # SOCK_DGRAM = UDP
servidor.bind(("0.0.0.0", PUERTO))
print("Eco UDP escuchando en el puerto", PUERTO, "...")
# OJO: no hay listen() ni accept(): en UDP nadie "acepta" nada

while True:
    # recvfrom devuelve DOS cosas: los datos Y la direccion del remitente
    datos, direccion = servidor.recvfrom(1024)
    mensaje = datos.decode("utf-8")                  # RECIBIR = DECODE
    print("Datagrama de", direccion, ":", mensaje)
    if mensaje.strip() == "":
        break
    eco = "ECO> " + mensaje
    # sendto necesita el DESTINO en cada envio, porque no hay conexion
    servidor.sendto(eco.encode("utf-8"), direccion)  # ENVIAR = ENCODE

servidor.close()
print("Eco UDP terminado.")
```

```python
#!/usr/bin/env python3
# ============================================================
#  ECO UDP (CLIENTE)
#  SIN connect(): cada datagrama lleva su destino.
#  EXPERIMENTO: manda un mensaje con el servidor APAGADO.
#  En UDP nadie avisa que se perdio; el timeout de 3 s
#  es lo que nos permite DETECTAR el silencio.
# ============================================================

import socket

DESTINO = ("localhost", 5050)   # o la IP del servidor

cliente = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# settimeout: la espera en recvfrom dura MAXIMO 3 segundos;
# si nadie responde, Python lanza socket.timeout
cliente.settimeout(3)
print("Eco UDP. Escriba mensajes (vacio para salir).")

while True:
    mensaje = input("Usted dice: ")
    cliente.sendto(mensaje.encode("utf-8"), DESTINO)   # ENVIAR = ENCODE
    if mensaje == "":
        break
    try:
        datos, _ = cliente.recvfrom(1024)              # RECIBIR = DECODE
        print("El servidor responde:", datos.decode("utf-8"))
    except socket.timeout:
        print("... silencio: el datagrama se perdio y NADIE aviso ...")

cliente.close()
print("Conexion (bueno, no habia conexion) terminada.")
```

**Cómo ejecutarlo (dos terminales) y el experimento de pérdida:**

1. **Terminal 1:** `python eco_udp_servidor.py` → `Eco UDP escuchando en el puerto 5050 ...`
2. **Terminal 2:** `python eco_udp_cliente.py` → escribe `hola` → llega `El servidor responde: ECO> hola`.
3. **Experimento:** en la terminal 1 detén el servidor con `Ctrl+C`. En la terminal 2 envía otro mensaje.
4. **Qué se observa:** tras 3 segundos, `... silencio: el datagrama se perdio y NADIE aviso ...`. No hubo error ni excepción de conexión: UDP no confirma entregas; solo el `settimeout` reveló el silencio.

## Código 5 — Framing con delimitador: la recepción correcta

Servidor que NO confía en que un `recv()` = un mensaje: acumula en un **buffer (búfer)** y corta por el delimitador `\n` (el byte 10, Tema 8).

```python
#!/usr/bin/env python3
# framing_servidor.py -- recepcion correcta con delimitador "\n"
# Problema que resuelve: TCP es un flujo de bytes; un recv(1024)
# puede traer mensajes PEGADOS o PARTIDOS.
import socket
```

```
PUERTO = 5061

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("localhost", PUERTO))
servidor.listen(1)
print("servidor framing> escuchando en", PUERTO)

conexion, direccion = servidor.accept()
print("servidor framing> cliente:", direccion)

buffer = ""            # aqui se ACUMULA lo que va llegando
while True:
    datos = conexion.recv(1024)
    if not datos:      # el cliente cerro la conexion
        break
    buffer += datos.decode("utf-8")
    # mientras haya un delimitador en el buffer, hay un mensaje COMPLETO
    while "\n" in buffer:
        mensaje, buffer = buffer.split("\n", 1)   # corta el primer mensaje
        print("servidor framing> mensaje completo recibido:", mensaje)

conexion.close()
servidor.close()
```

```
#!/usr/bin/env python3
# framing_cliente.py -- envia DOS mensajes en UN solo send()
import socket

cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("localhost", 5061))

# dos mensajes viajan JUNTOS en el mismo envio
cliente.send("primer mensaje\nsegundo mensaje\n".encode("utf-8"))
cliente.close()
```

**Salida real verificada:** el servidor imprime DOS líneas (`mensaje completo recibido: primer mensaje` y `... segundo mensaje`), aunque los bytes llegaron juntos. Sin buffer ni delimitador, un solo `recv()` los habría tratado como una sola cosa.

**Cómo ejecutarlo (dos terminales):**

1. **Terminal 1:** `python framing_servidor.py` → queda escuchando en el puerto 5061.
2. **Terminal 2:** `python framing_cliente.py` → envía los dos mensajes juntos y termina.
3. **Qué se observa:** la terminal 1 imprime **dos** líneas separadas — el buffer + el delimitador `\n` separaron lo que llegó pegado.

## SEGUNDA PARTE — EJERCICIOS (para imprimir y contestar a lápiz)

**Nombre:** _____ **Equipo:** _____ **Fecha:** _____

### Sección A0 — Repaso de Python (encierra la letra)

**A0.1.** En Python, `PUERTO = 5050` crea:

a) una función b) una variable de tipo entero c) una librería d) un socket

**A0.2.** `b"hola"` es un valor de tipo:

a) `str` b) `int` c) `bytes` d) `bool`

**A0.3.** El ciclo `while True:` termina cuando:

a) llega a 10 vueltas b) se ejecuta un `break` c) se acaba la memoria d) nunca puede terminar

**A0.4.** `conexion, direccion = servidor.accept()` funciona porque `accept()` devuelve:

a) una lista b) un diccionario c) una tupla de dos elementos d) un bool

**A0.5.** Un entorno virtual (venv) sirve para:

a) hacer Python más rápido b) aislar las librerías y versiones de cada proyecto c) conectarse a internet d) cifrar el código

### Sección A — Opción múltiple (encierra la letra)

**A1.** Un sistema distribuido se caracteriza porque sus componentes:

a) comparten la misma memoria RAM b) están en computadoras conectadas por red y se comunican solo por paso de mensajes c) corren en la misma máquina d) no necesitan protocolos

**A2.** En el modelo cliente-servidor:

a) el servidor inicia la comunicación b) el cliente inicia y el servidor espera en una dirección conocida c) ambos inician al mismo tiempo d) no hay direcciones conocidas

**A3.** ¿Cuántos bytes puede representar un valor de 8 bits (un byte)?

a) 8 b) 127 c) 256 d) 1024

**A4.** El número ASCII 10 corresponde a:

a) el espacio b) la letra A c) el salto de línea (`\n`) d) el dígito 0

**A5.** El puerto bien conocido de HTTPS es:

a) 80 b) 22 c) 443 d) 53

**A6.** La llamada `accept()` devuelve:

a) solo la dirección del cliente b) un socket nuevo para hablar con ese cliente y su dirección c) el número de clientes conectados d) el backlog disponible

**A7.** Los tres mensajes del handshake TCP, en orden:

a) ACK, SYN, SYN-ACK b) SYN, SYN-ACK, ACK c) SYN, ACK, FIN d) HELLO, SYN, ACK

**A8.** `recv(1024)` significa:

a) recibe exactamente 1024 bytes b) recibe exactamente un mensaje c) recibe HASTA 1024 bytes de lo que haya llegado d) recibe 1024 mensajes

**A9.** En UDP, la dirección del remitente se obtiene con:

a) `accept()` b) `connect()` c) `recvfrom()` d) `bind()`

**A10.** Para enviar un ARCHIVO de 3 MB de forma confiable, el transporte adecuado es:

a) UDP, porque es más rápido b) TCP, porque garantiza entrega y orden c) da igual d) ninguno, los archivos no viajan por red

**A11.** Para video EN TIEMPO REAL, el transporte adecuado es:

a) TCP, porque retransmite los cuadros perdidos b) UDP, porque un cuadro perdido retransmitido llegaría tarde c) da igual d) Telnet

**A12.** El propósito principal de RPC es:

a) cifrar mensajes b) hacer la red más rápida c) transparencia de acceso: llamar lo remoto como si fuera local d) eliminar el sistema operativo

## Sección B — Verdadero o falso (encierra y justifica en la línea)

**B1.** En un sistema distribuido, una parte puede fallar mientras las demás siguen funcionando; tu programa debe contemplarlo. **V / F**

---

**B2.** La memoria compartida es el modelo natural para comunicar máquinas distintas por red. **V / F**

---

**B3.** 1 KB equivale a 1000 bytes exactamente. **V / F**

---

**B4.** El carácter ñ ocupa exactamente 1 byte en UTF-8. **V / F**

---

**B5.** El socket que devuelve `accept()` es el mismo que hizo `listen()`. **V / F**

---

**B6.** TCP garantiza que cada `send()` corresponde a exactamente un `recv()`. **V / F**

---

**B7.** Si un datagrama UDP se pierde, el protocolo lo retransmite automáticamente. **V / F**

---

**B8.** UDP es ideal para aplicaciones que no pueden permitir la pérdida de datos. **V / F**

---

## Sección C — Relacionar columnas (escribe la letra)

| Llamada | Tu respuesta | Descripción |
|---|---|---|
| 1. `socket()` | _____ | a. Asocia el socket a una dirección y puerto locales |
| 2. `bind()` | _____ | b. Inicia la conexión hacia el servidor (dispara el handshake) |
| 3. `listen()` | _____ | c. Crea el punto final de comunicación |
| 4. `accept()` | _____ | d. Envía un datagrama indicando el destinatario cada vez |
| 5. `connect()` | _____ | e. Pone el socket en modo escucha |
| 6. `recvfrom()` | _____ | f. Cierra el punto final de comunicación |
| 7. `sendto()` | _____ | g. Recibe un datagrama y la dirección de quien lo envió |
| 8. `close()` | _____ | h. Acepta una conexión y devuelve un socket nuevo + dirección |

## Sección D — Respuesta corta (usa las líneas)

**D1.** Define con tus palabras "sistema distribuido" y menciona DOS problemas que no existen en una sola máquina.

**D2.** Explica la regla de oro "ENVIAR = ENCODE · RECIBIR = DECODE" y qué error aparece si se olvida cada mitad.

**D3.** ¿Por qué el servidor necesita `bind()` y el cliente no? ¿Qué hace el sistema operativo por el cliente?

**D4.** Explica el problema del framing y describe DOS soluciones.

**D5.** ¿Qué son stub y skeleton en RPC, en qué máquina va cada uno, y qué es *marshalling*?

## Sección E — Lectura de código

**E1.** ¿Qué imprime exactamente este programa cuando un cliente se conecta y envía `"hola"`?

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.bind(("localhost", 5050))
servidor.listen(1)
conexion, direccion = servidor.accept()
datos = conexion.recv(1024)
print("Me llego:", datos)
print("Tipo:", type(datos))
```

---

---

**E2.** Este código UDP tiene UN error conceptual. Encuéntralo y corrígelo:

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
servidor.bind(("0.0.0.0", 5050))
servidor.listen(1)                          # linea con problema
datos, direccion = servidor.recvfrom(1024)
```

---

---

**E3.** ¿Cuál es el error de este cliente TCP?

```
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("localhost", 5050))
cliente.send("hola servidor")
```

---

---

## Sección F — Ejercicios prácticos (en tu computadora; entrega capturas)

Cada ejercicio usa los **pasos de ejecución en dos terminales** del código correspondiente (Tema 20). Verifica la salida esperada antes de contestar.

**F1. Ciclos de vida (Códigos 1 y 2).** Ejecuta el eco TCP. En una copia del código, subraya:

(a) la línea donde el servidor se bloquea esperando cliente; (b) la línea donde se bloquea esperando datos; (c) la línea del cliente que dispara el handshake.

**F2. Ciclo `while True` + funciones (Código 1).** Modifica el servidor de eco: agrega un contador y una función `armar_eco(mensaje, numero)` que devuelva `ECO #N> mensaje`. Prueba con tres mensajes.

**F3. UDP y best-effort (Código 4).** Ejecuta el experimento de pérdida (apagar el servidor a la mitad). Documenta con captura qué observó el cliente y explica en 3 líneas por qué no hubo ningún error de conexión.

**F4. Framing (Código 5).** Ejecuta el servidor y cliente de framing. Luego modifica el cliente para enviar TRES mensajes en un solo `send()` y verifica que el servidor los separa. ¿Qué línea del servidor es la que "sabe" dónde termina cada mensaje?

**F5. Bloqueo (Código 3).** Cambia `ESPERA` a 7 segundos, ejecuta y anota el tiempo que midió el servidor. Después agrega `conexion.settimeout(5)` justo después de `accept()` y vuelve a correr: ¿qué excepción ocurre y en qué lado?

**F6. Código limpio.** Reescribe este código cumpliendo TODAS las normas de código limpio (constantes, nombres, comentarios que explican el porqué):

```
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.bind(('',9000))
s.listen(1)
c,a=s.accept()
d=c.recv(1024)
print(d)
```

## Referencias


**Principales (en el orden de las unidades del curso):**

- **López-Fuentes, F. (2015).** *Sistemas Distribuidos*. UAM — sec. 4.5: modelos de comunicación (paso de mensajes, cliente-servidor, RPC). **Base de la Unidad I.**
- **Rhodes, B. & Goerzen, J. (2014).** *Foundations of Python Network Programming*. 3.ª ed., Apress — cap. 1 (introducción a sockets), cap. 2 (UDP), cap. 3 (TCP), cap. 5 (framing y manejo de datos). **Base de la Unidad II.**

**De apoyo:**

- Coulouris, G., Dollimore, J., Kindberg, T. & Blair, G. *Distributed Systems: Concepts and Design* — cap. 1 (sistemas distribuidos, middleware, transparencia), cap. 4 (comunicación entre procesos).
- Kurose, J. & Ross, K. *Computer Networking: A Top-Down Approach* — cap. 1 (protocolos) y cap. 3 (capa de transporte: TCP, handshake, UDP).
- Stevens, W. R., Fenner, B. & Rudoff, A. (2004). *Unix Network Programming*, vol. 1 — referencia clásica de la API de sockets Berkeley.
- Tanenbaum, A. & Van Steen, M. *Distributed Systems* — definición alternativa de sistema distribuido.
- Documentación oficial: `socket` — https://docs.python.org/3/library/socket.html · Git — https://git-scm.com/doc

## HOJA DE RESPUESTAS

*(Sección para el maestro: retirar antes de imprimir los ejercicios.)*

### Sección A0

- **A0.1. b** — es una variable `int`; por convención del curso las constantes van en mayúsculas.
- **A0.2. c** — la `b` inicial indica `bytes`: lo único que viaja por la red.
- **A0.3. b** — `while True` es infinito; solo un `break` (o cerrar el programa) lo detiene.
- **A0.4. c** — devuelve una tupla `(socket_nuevo, direccion)` y se desempaca en dos variables.
- **A0.5. b** — cada proyecto con sus librerías, sin interferir con otros.

### Sección A

- **A1. b** — es la definición de Coulouris: componentes en red comunicándose por mensajes.
- **A2. b** — servidor pasivo y conocido; cliente activo.
- **A3. c** — $2^8 = 256$ valores (0 a 255).
- **A4. c** — el byte 10 es el salto de línea (line feed); por eso `\n` baja una línea.
- **A5. c** — 443 (el 80 es HTTP sin cifrar).
- **A6. b** — socket nuevo de conversación + tupla (IP, puerto) del cliente.
- **A7. b** — SYN, SYN-ACK, ACK.
- **A8. c** — "hasta 1024 bytes"; TCP no conoce mensajes.
- **A9. c** — `recvfrom()` devuelve (datos, dirección); UDP no tiene `accept()`.
- **A10. b** — un archivo debe llegar completo y en orden: TCP.
- **A11. b** — en tiempo real un cuadro retrasado no sirve: UDP.
- **A12. c** — RPC busca que lo remoto se sienta local.

### Sección B

- **B1. Verdadero** — es el "fallo parcial", rasgo definitorio de los sistemas distribuidos.
- **B2. Falso** — la memoria compartida exige misma máquina; en red el modelo es paso de mensajes.
- **B3. Falso** — 1 KB = 1024 bytes (potencias de 2).
- **B4. Falso** — en UTF-8, `ñ` ocupa 2 bytes; solo los 128 caracteres ASCII ocupan 1.
- **B5. Falso** — `accept()` devuelve un socket NUEVO; el de escucha sigue solo aceptando.
- **B6. Falso** — TCP es un flujo de bytes: los mensajes pueden llegar pegados o partidos (de ahí el framing).
- **B7. Falso** — UDP es best-effort: sin retransmisión ni aviso.
- **B8. Falso** — al revés: UDP es para aplicaciones que TOLERAN pérdida; TCP garantiza entrega.

### Sección C

1 → c · 2 → a · 3 → e · 4 → h · 5 → b · 6 → g · 7 → d · 8 → f

### Sección D (respuestas modelo)

- **D1.** Componentes en computadoras conectadas por red que se comunican por paso de mensajes y aparecen ante el usuario como un solo sistema. Problemas nuevos: latencia, fallos parciales, pérdida/desorden de mensajes, concurrencia, heterogeneidad (dos cualesquiera).

- **D2.** Los sockets solo transportan bytes; el texto se convierte a bytes al enviar (`encode("utf-8")`) y se reconstruye al recibir (`decode("utf-8")`). Olvidar el encode: `TypeError`; olvidar el decode: se imprime `b'hola'`.
- **D3.** El servidor necesita una dirección conocida y fija para que lo encuentren: eso hace `bind()`. Al cliente nadie lo busca: al hacer `connect()`, el sistema operativo le asigna un puerto efímero automático.
- **D4.** TCP entrega un flujo de bytes sin fronteras: un `recv()` puede traer un mensaje partido o varios pegados. Soluciones: (1) delimitador (`"\n"`, el byte 10) acumulando hasta encontrarlo; (2) prefijo de longitud (enviar primero cuántos bytes vienen); ambas con ciclo de recepción.
- **D5.** Stub: representante local de la función remota (máquina CLIENTE): empaqueta, envía, devuelve. Skeleton: lado SERVIDOR: desempaqueta, ejecuta la función real, empaqueta el resultado. *Marshalling*: convertir los datos a un formato transmisible por la red.

## Sección E

- **E1.** Imprime `Me llego: b'hola'` y `Tipo: <class 'bytes'>`. La `b` delata que `recv()` devuelve BYTES; faltó `datos.decode("utf-8")`.
- **E2.** `listen()` **no existe en UDP**: no hay conexiones que escuchar. Se elimina; el servidor UDP correcto es `socket → bind → recvfrom/sendto → close`.
- **E3.** `send()` con texto sin codificar: debe ser `cliente.send("hola servidor".encode("utf-8"))`. Tal como está, lanza `TypeError: a bytes-like object is required`.

## Sección F (orientaciones)

- **F1.** (a) `conexion, direccion = servidor.accept()`; (b) `datos = conexion.recv(1024)`; (c) `cliente.connect((IP_SERVIDOR, PUERTO))`.
- **F2.** `contador = 0` antes del `while`; función:

```python
def armar_eco(mensaje, numero):
    return "ECO #" + str(numero) + "> " + mensaje
```

- **F3.** El cliente imprime `... silencio: el datagrama se perdio y NADIE aviso ...` tras 3 s. No hay error porque UDP no confirma entregas: el datagrama se fue "al vacío" y solo el `settimeout` reveló el silencio.
- **F4.** La línea es `while "\n" in buffer:` (junto con el `split("\n", 1)`): el delimitador marca dónde termina cada mensaje.
- **F5.** El servidor mide ≈ 7 s. Con `settimeout(5)`, a los 5 s salta la excepción `socket.timeout` **en el servidor** (el lado que espera); se captura con `try/except socket.timeout`.
- **F6.** Solución modelo:

```python
PUERTO = 9000

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("localhost", PUERTO))
servidor.listen(1)

# accept() devuelve un socket NUEVO para hablar con este cliente
conexion, direccion = servidor.accept()
datos = conexion.recv(1024)
print("Me llego:", datos.decode())
```